BENCHMARK REPORT

Native Movement Response and Validation

Matched-state response, dot-core isolation, and camera-to-display evidence.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Experimental X/Y correction investigation - September 7, 2026

The UNO dot comparison rejected the initial optical-flow path: only 17 of 128 observed still-hold status samples were detected, versus 177/177 in the original MediaPipe baseline. Inspection found that correction replayed every intervening frame, then submitted the next recognition job using an already-aged source.

A synthetic before/after benchmark ran on the UNO's installed Python/OpenCV environment (OpenCV 4.10.0, four OpenCV threads), with camera processing paused. It used moving textured images at requested 60 Hz and a simulated 60 ms recognition delay, with no camera access, controller transmission or game input. Two alternating four-second runs per version produced:

Measure	Original correction	Revised correction
Foreground processing p95	78.3 / 80.0 ms	18.6 / 18.3 ms
Valid synthetic frames	71/139 / 69/139	231/235 / 229/233
Recognition-source age p95	265.9 / 266.1 ms	130.6 / 130.1 ms

The revised runs rejected only their four startup frames. The fix performs one checked source-to-current correction on at-most-320-pixel-wide flow images and submits the next recognition before correction. Original recognition resolution, the 250 ms freshness limit, and per-player reach remain unchanged. Correction over 25 ms is rejected; an individual OpenCV call is not interruptible. New diagnostics separate correction, completed-result pickup and rejection reasons.

The simulated recognizer sleeps rather than competing for CPU like MediaPipe. This proves a correction-cost improvement on the target hardware, not live recognition reliability or physical hand-to-screen latency. Large movements or low palm texture may still fail the optical-flow checks. The revised code is installed with experimental mode **off**; the original MediaPipe path remains active pending another operator-cued hand test.

Reproduce with the original `motion.py` saved outside the checkout, using an environment containing the vision dependencies:

```
SH
PYTHONPATH=src python3 scripts/benchmark-motion-correction.py \
  --before /tmp/original-motion.py --output /tmp/correction-comparison.json
```

Run while normal camera processing is idle. The script never changes controller settings itself. The original UNO report is retained locally in `/tmp/uno-dot-baseline-x8heur1f/correction-benchmark-uno.json`.

The **PowerGlove Vision Controller (Arduino UNO Q)** performs the camera, recognition, and send stages measured in this record.

This deterministic headless benchmark compares the same exact Super Glove Ball ROM through its native packet path and its conventional FCEUmm joystick path. Gun Smoke remains available as an optional positional-FCEUmm reference. This is a software validation tool, not a substitute for camera, display, or physical cabinet testing.

Reproducible inputs

Lane	Core	Exact game image
Native coordinates	`Nestopia PowerGlove` built from pinned Nestopia revision `5a1cd378cb46ca9ccc2dd6f8b2b6a79ab986052e` plus the repository patch	`Super Glove Ball (USA)`, SHA-256 `ad60ef1b62cd1b3bc02a9320376067347a8ab2ebbe46e1616693d8379c9d9a7b`
Standard joystick	Stock `FCEUmm` revision `236ccdfc911e84c60fea6b9d0699c2d440a8de14`	The same exact `Super Glove Ball (USA)` image
Optional standard-D-pad reference	The same stock `FCEUmm` revision	`Gun Smoke (USA)`, SHA-256 `4ad9629a2bacc158a7f50975869c7dfe533567ae399a5bdc5df2240286df259f`

ROMs stay outside the project and every result is tied to its digest. Gun Smoke uses the positional Program G mapping, making it a useful second FCEUmm exercise of the shared camera-direction recognition when supplied.

Direct-output dot test

Use the installed `lr-powerglove-dot` core when game behavior makes native X/Y difficult to judge. It reuses the normal Controller camera, MediaPipe recognition, reach mapping, signed transport, receiver, and 64-byte native-state publication; the dot core replaces only the game's interpretation and ignores the ROM.

1. Launch Super Glove Ball through the normal cabinet menu and select `lr-powerglove-dot` for that launch. Keep the normal start/end hooks and Controller game session.
2. Select the intended player, start controller delivery, confirm calibration, and close Dashboard preview. A yellow dot and `TRACKING` should follow the hand. Leaving view must show `NO INPUT`; returning must restore the dot.
3. Film the hand and display together at a verified high frame rate. Hold still for five seconds, make three horizontal and three vertical movements with one-second holds, then test loss and recovery separately.
4. Exit normally so receiver and core traces finalize. Repeat the same motion with `lr-nestopia-powerglove` and unchanged player and camera settings.

For guided software measurements, run `scripts/run-native-latency-session.py` on the cabinet. It collects bounded, read-only native telemetry and does not launch a game, start camera output, or alter calibration. Synthetic publication proves transport behavior only; it is not evidence of physical hand-to-display latency.

Method

The runner boots each exact ROM into play and saves one emulator state. For each direction it restores that same state twice: once for the baseline and once with only the candidate input changed. It records the first emulated video frame whose checksum differs. Release uses the same comparison after eight frames of held input: continuing the hold is the baseline and returning to neutral is the sole change.

FCEUmm also records whether the libretro input callback was polled on the first changed frame and which input-device API it requested. For Super Glove Ball it requested only libretro device `1`, the standard joypad, confirming that the native state file is not part of that lane. The native lane publishes one coherent state immediately before every emulated frame; the custom core's packet trace is the separate evidence that this state is sampled once per frame. The shared recognition check proves all four directions activate at `0.29` normalized displacement and release at `0.13`, on the responsive side of the configured `0.28`/`0.14` boundaries. It also drives the bounded native coordinate curve directly. Motion speed is

calculated from consecutive raw selected coordinates and capture timestamps, normalized by directional calibrated reach and combined into one vector follow weight. An elliptical jitter region separates rest from intent. A large X/Y change must follow the newest sample immediately, medium travel must reach at least 90% within 150 ms, resting jitter must remain bounded, and stops or reversals must leave no catch-up tail or overshoot.

Results

Three complete executions produced byte-identical reports.

Lane	Directions	First visible activation	First visible release	First-frame input pickup
Super Glove Ball / `lr-nestopia-powerglove`	Left, right, up, down	Frame 3, about 50.0 ms at 60 Hz	Frame 3, about 50.0 ms at 60 Hz	Native state published before frame; per-frame packet consumption established by the trace runner
The same Super Glove Ball ROM / stock FCEUmm	Left, right, up, down	Frame 3, about 50.0 ms at 60 Hz	Frame 3, about 50.0 ms at 60 Hz	Standard joypad callback polled on frame 1; no native packet input
Gun Smoke / stock FCEUmm reference	Left, right, up, down	Frame 2, about 33.3 ms at 60 Hz	Frame 2, about 33.3 ms at 60 Hz	Standard joypad callback polled on frame 1

The native positive-X sweep also diverged on frame 3 at every tested magnitude: `1024`, `2048`, `4096`, `8192`, `16384`, and `32767`. The smallest step is about 3.1% of the positive signed coordinate range, so the test demonstrates continuous small-motion response rather than only edge-to-edge movement.

The same-ROM visual comparison exposed and corrected a native Y-axis wrapping error that a checksum-only test had missed. The corrected trace now returns `$80`, `$00`, and `$7F` for Y minimum, center, and maximum, and the corresponding screens place the Robo-Glove at bottom, center, and top. Conventional FCEUmm directions also reach their matching screen edges, but they do so as held digital commands; native coordinates specify an absolute target position. This is the substantive gameplay difference between the two modes even though their first visible response occurs on the same emulated frame in this ROM.

These numbers are the first input-caused *visible* frame, not a camera-to-screen wall-clock claim. Camera capture cadence, the 75 ms Academy polling interval, display buffering, and physical display latency are intentionally outside this headless core benchmark.

The Dashboard now reports rolling camera-read-to-send and changed-control-to-send p50/p95 measurements. Those cover the PowerGlove Vision Controller software stage for both FCEUmm and native X/Y. Receiver publication and the core's next-frame consumption remain separate stages: the coherent native record timestamps publication on RetroPie after packet validation and virtual-gamepad writes, rather than socket arrival, and the headless core benchmark publishes the changed record immediately before an emulated frame.

The live native path now uses completed MediaPipe palm coordinates exclusively. The Dashboard can compare the bounded speed curve with direct latest coordinates; both use the same reach mapping and safety behavior. The optical-flow lane was reported as jerky and unreliable and is archived for source-level and historical trace comparison rather than exposed as a runtime option.

Repeatable camera comparison

`scripts/record-vision-benchmark.py` records a temporary local 30-second cue sequence containing full-field and short movement, neutral jitter, A/B, rolls, Closed Hand, push, pull, tracking recovery, and near/far poses. The companion `scripts/benchmark-vision-replay.py` runs the same full frames through MediaPipe Hands with 1, 2, and 4 inference threads, plus MediaPipe Tasks Video when its model is supplied. Each runs at 640×480 and a full-field 512×384 resize with preview work both closed and open; neither resolution crops the image.

The replay report includes inference p50/p95, detection continuity, cue recognition, first recognition within each cue, neutral false activations, coordinate jitter, and preview cost. Live Dashboard readings remain authoritative for latest-camera-frame age because an offline replay has no live capture queue. The temporary AVI and cue sidecar must be deleted after aggregate conclusions are retained. The proven 640×480 configuration remains the default unless a candidate improves p95 by at least 15%, introduces no neutral false activation, meets the response targets, and loses no more than one percentage point of labeled recognition.

For a user-paced capture, `scripts/guided-vision-benchmark.py` serves a temporary live camera page. Nothing is recorded while the player frames a pose. Selecting **Record this step** starts a two-second countdown and records only that labeled step; the player decides when to continue. The completed camera source remains local and releases the camera automatically. A fixed-duration subset may then be sampled from those confirmed steps for repeatable replay. Guided capture is diagnostic evidence, not training data or an automatic part of Glove Academy.

Preliminary PowerGlove Vision Controller steady-state timing

After deploying 0.3.2-dev on September 5, 2026, two controller-off 300-sample smoke-test windows exercised the current MediaPipe Hands configuration at 640×480. These validate the camera, inference, state calculation, telemetry, and preview isolation; they do not replace the labeled clip or console tests.

Preview state	Recognition rate	Inference p50 / p95	Camera read to decision p50 / p95
Dashboard closed	11.4 Hz	87.9 / 98.4 ms	109.6 / 127.9 ms
Dashboard stream open	11.8 Hz	88.0 / 97.6 ms	108.2 / 125.7 ms

The preview-open window did not materially increase p95. Controller delivery was intentionally stopped, so no changed-control-to-send samples were recorded; that metric requires the authenticated RetroPie receiver during the live test.

Guided replay result - September 5, 2026

A player-confirmed guided session recorded 1,575 frames across 52 seconds at an effective 30.29 fps. A 30-second comparison subset retained the first two seconds of every labeled step at 15 fps, giving all 16 lanes the same 450 source frames and 67 ms temporal resolution. The complete guided source was preserved while the comparison ran.

Backend and full-frame size	Threads	Preview	Inference p50 / p95	Detection continuity	Neutral false frames
MediaPipe Hands, 640×480	1	Closed	96.51 / 207.60 ms	71.33%	14
MediaPipe Hands, 640×480	2	Closed	92.94 / 217.40 ms	71.33%	14

Backend and full-frame size	Threads	Preview	Inference p50 / p95	Detection continuity	Neutral false frames
MediaPipe Hands, 640×480	4	Closed	98.60 / 190.18 ms	71.33%	14
MediaPipe Tasks Video, 640×480	1	Closed	184.55 / 403.13 ms	82.89%	44
MediaPipe Hands, 512×384	4	Closed	97.37 / 213.53 ms	72.67%	60
MediaPipe Tasks Video, 512×384	1	Closed	183.85 / 401.05 ms	81.11%	43
MediaPipe Hands, 640×480	4	Open	99.45 / 190.36 ms	71.33%	14
MediaPipe Tasks Video, 640×480	1	Open	187.21 / 382.52 ms	82.89%	44

Tasks Video recovered difficult A, B, and Closed Hand poses that the proven backend missed in the selected windows, but approximately doubled median inference time and exceeded the gameplay latency target decisively. The 512×384 resize did not improve p95 by the required 15%; it increased neutral false activations and weakened roll recognition. Thread count did not change recognition, and no alternative produced a consistent qualifying latency gain. No replay alternative met the 15% promotion threshold in that early clip. Subsequent matched live gameplay did promote **MediaPipe Hands** at 640×480 with four explicitly selected inference threads for 0.4.0. That later selection also uses a `0.40` tracking-confidence threshold and a 30-fps-first camera policy. Preview encoding at full size measured about 9.6 ms p95 in this historical run; 0.4.0 moves gameplay annotation and 320×240 encoding off the inference thread.

The replay deliberately saturates inference and produced higher tail latency than real-time capture. The live steady-state p95 measurements above remain the authoritative gameplay-stage values. Replay is used for relative comparisons and identical-frame recognition evidence.

The guided source also documented a difficult backlit scene. Mean luma was approximately 77-79 on a 0-255 scale and 34-38% of pixels were below luma 32. The camera was already using automatic exposure. Its hardware backlight compensation made a reversible still-image test slightly darker, while restoring brightness and contrast to factory defaults helped only modestly. No camera control was promoted globally. Front lighting or moving the bright window out of the background is the safer remedy because forced exposure can add motion blur and reduce the stable frame rate.

Collect a live status baseline

Before changing responsiveness settings, measure one stage at a time. Preserve the same camera position, lighting, calibration, recognition settings, and game state between comparisons. The read-only collector does not activate the camera or enable controller delivery; prepare the intended mode on Dashboard first.

For a stationary open-hand window, run from the development checkout:

```
SH
python3 scripts/measure-vision-status.py \
  --status-url http://UNO-Q-NAME.local:8088/status \
  --phase neutral --seconds 30 --output /tmp/powerglove-neutral.json
```

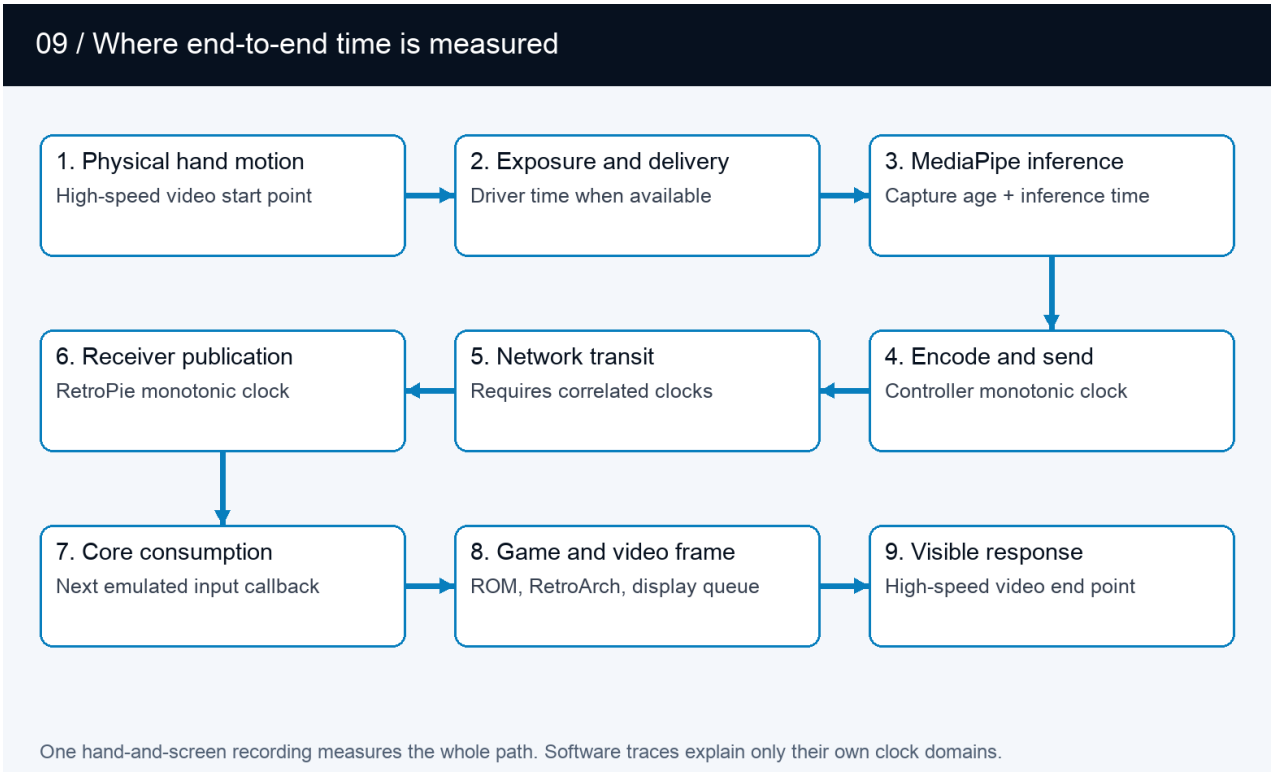
Repeat with `--phase movement` and a new output path while making deliberate short X/Y steps and returns. Use a separate run for preview-open and preview-closed conditions. Neither run records camera images. The neutral report

includes observed signed-axis span and standard deviation; those combine physical hand movement with tracker variation and are not an isolated sensor noise measurement. The phase label describes the operator's test, not an automatically verified pose. Do not label ordinary gameplay as a neutral test.

The collector counts each observed inference timestamp/capture-sequence pair once, rejects invalid timings, and separates changing profiles, delivery gates, preview-client counts, and camera/backend settings. It reports p50/p95 over unique observations, not averages of the worker's overlapping rolling windows. Detected-hand and missing-hand inference distributions remain separate, and each segment reports the change in the worker's skipped-capture counter. Public status is cached by the supervisor, so these are sampled distributions, not a complete frame trace. A faster polling interval cannot recover frames that were never exposed by that cache. A direct worker `/status` read inside its container avoids the supervisor cache but still samples results.

Review fresh-sample counts, request errors, detection/calibration counts, and local send-success counts before comparing runs. `sent_sample_age_ms` contains only locally successful sends, whereas ordinary `sample_age_ms` also exists with delivery stopped. Neither proves receiver acceptance. An idle window produces no active samples and exits with code 2; it must not be presented as a zero-latency result. The report excludes addresses, tokens, images, landmarks, and individual coordinate records. Full options are in the Configuration Reference.

Keep the stage boundaries separate



End-to-end latency boundaries from the physical hand and camera through MediaPipe, RetroPie, the emulator, and the visible game

Stage	Evidence to collect	What it does not establish
Exposure and camera delivery	Physical visual reference plus camera/driver timestamps when available	OpenCV read-completion timestamps do not measure exposure or upstream buffering.

Stage	Evidence to collect	What it does not establish
Capture to inference	`capture_age_ms`, processed capture spacing, negotiated camera mode, and frame skips	Processed-frame spacing is not the spacing of every camera frame.
Inference and recognition	`inference_ms`, inference spacing, detection continuity, and fixed-input replay	Existing inference timing includes tracking and gesture work; replay is not live capture.
Local send	`send_ms`, successful-send counts, and `sent_sample_age_ms`	Successful UDP submission is not a delivery acknowledgement.
Network reception	Controller-to-console round trips as a diagnostic; correlated receive timestamps for actual UDP measurements	ICMP round trips cannot be relabeled as one-way gameplay delivery time.
Receiver publication	Local timestamps at receive, after validation/uinput, and after publishing the even guard	The current record timestamp starts at publication; it hides preceding receiver work.
Native core consumption	Match a published sample to the core callback using the console's monotonic clock	Polling the native file from another process does not prove when the core consumed it.
Game and display	Exact-ROM frame response, active RetroArch video settings, and a high-frame-rate hand/screen recording	The headless frame-3 result excludes presentation buffering and physical display response.

Use durations within one machine's clock domain. Do not subtract independent monotonic clocks across machines, halve an ICMP round trip into a claimed UDP delay, or add independently measured p95 values into an end-to-end percentile. The opt-in tools in the native session procedure provide receiver/core instrumentation. Collect actual traces and compare their overhead before reporting those intervals as measured.

How the camera experiments fit the complete path

The camera comparisons answer whether a fresher image reaches MediaPipe; they do not measure the complete controller. Their results belong beside, not in place of, model, transport, emulator, and display evidence.

Experiment	Boundary covered	Result	Still outside the measurement
Kiyo 640x480 and 1280x720, one or two OpenCV buffers, HDR state comparison	Camera delivery and JPEG decode before inference	640x480 with fixed-rate automatic exposure and HDR off produced the most consistent tested capture lane	MediaPipe, gestures, network, emulator, display
Automatic, 30 fps, and 60 fps live play	Negotiated camera cadence plus the complete chain as perceived by the player	30 fps felt smoother and more attached than the 60-fps request and became the first choice	No synchronized physical latency number
Direct V4L2 at 640x480 MJPEG and 30 fps	Driver timestamp to newest-buffer dequeue	Worked on the Kiyo Pro; a representative fresh frame measured about 1.25 ms and 33.3 ms capture cadence	Exposure start, MediaPipe, transport, game, display
OpenCV versus direct V4L2	Alternate entrance to the same newest-frame slot	Direct capture passed on the test camera and remains optional with automatic fallback	Does not change the MediaPipe model or game mapping
Full versus lean MediaPipe graph	Model output and resulting sample age after capture	Lean median inference improved only about 1.6% and p95 sample age worsened, so the full graph remained selected	Network, core, rendered response
Dashboard closed versus open	Optional preview and statistics load around the Controller path	Preview uses latest-only work; routine statistics are omitted unless requested	Does not isolate camera exposure or display latency

Functional live play, including a completed Super Glove Ball game, proves that the entire chain can work together. It does not assign delay to an individual stage. The remaining definitive latency test is a repeated high-frame-rate view of the physical hand and screen, correlated with Controller, receiver, and core traces from the same session.

For the visual test, frame the hand and game display in one 120/240-fps recording. Hold an open hand still for five seconds, then make five short horizontal steps with a pause after each. Count from first physical motion to first corresponding game motion for each step, recording the camera frame rate and uncertainty. Also inspect resting movement and repeats in the reverse direction. This measures the complete visible path; it does not by itself assign delay to one software stage.

September 6 diagnostic preflight

With the cabinet newly booted, a 20-packet Controller-to-RetroPie ICMP probe returned all packets: minimum 0.315 ms, average 0.389 ms, maximum 0.666 ms, and reported deviation 0.088 ms. This short idle-network check is not a gameplay UDP latency measurement. The cabinet reported 1920x1080 at 60 Hz, threaded video enabled in the global configuration, and no reported throttling. Effective per-game settings and live timing must still be checked with the game running. No recognition, smoothing, camera, network, or video defaults were changed.

The subsequent game launch confirmed `lr-nestopia-powerglove`, device `517`, and the native-state path. Both active append files were checked for video overrides; they added a 60.00 Hz refresh value and did not override threaded video. The user confirmed the Robo-Glove followed their hand before repeating the two windows below. Measurements came directly from the worker at a 50 ms poll interval, with the proven 640x480 MJPG backend, delivery enabled, and one preview client. Camera metadata reported a requested/negotiated 60 fps; this is not proof of a 60 fps effective capture rate.

Window	Fresh / detected observations	Detected inference p50 / p95	Camera-read-to-send p50 / p95
Requested stationary hold, 10 seconds	91 / 78	91.1 / 109.9 ms	116.6 / 205.4 ms
Short movement, 30 seconds	315 / 315	91.0 / 103.2 ms	112.3 / 132.8 ms

Both windows had zero request errors and locally successful sends for every observed sample. The stationary attempt included 13 tracking misses, whose inference measured 175.7 / 204.5 ms p50/p95. Its X/Y spans were 25,080 / 17,829 signed-axis units, so it is not an accepted stationary-jitter baseline without video review and a repeat. The movement window retained detection throughout. Superseded-capture counter deltas were 205 and 584 respectively; intentional frame replacement is not network packet loss.

A separate 15-second read-only native-file observer saw 128 distinct guarded publications, including 101 detected/calibrated Super Glove Ball samples, and rejected two incoherent reads. For those gameplay samples, observed publication interval p50/p95 was 94.139 / 109.235 ms. Publication-to-observer age p50/p95 was 1.105 / 2.154 ms. This was a different window with a 2 ms polling sleep: it does not measure socket arrival, publication cost, or core pickup, and it can miss overwritten records. It is cadence evidence only.

These results identify inference and tracking recovery as substantial measured costs. They do not yet justify changing stabilization, camera settings, or threaded video. Frame-by-frame video analysis, a reliable stationary window, receiver timing instrumentation, and native-consumption timing remain outstanding. Read-to-send timing also omits the wait for the next inference opportunity when physical motion begins between processed frames.

Run it again

Build the two isolated benchmark cores:

```
SH
scripts/build-nestopia-powerglove.sh
scripts/build-fceumm-benchmark.sh
```

Then supply external paths to the exact ROMs:

```
SH
python3 scripts/benchmark-direction-response.py \
  --nestopia-core build/nestopia-powerglove/nestopia_powerglove_libretro.so \
  --super-glove-ball-rom "/path/to/Super Glove Ball (USA).nes" \
  --fceumm-core build/fceumm-benchmark/fceumm_libretro.so \
  --scratch /tmp/powerglove-direction-benchmark \
  --output /tmp/powerglove-direction-benchmark/result.json
```

On macOS, use the emitted `.dylib` paths instead of `.so`. Build products, scratch state, reports, and ROMs are not release-package content. The runner's `--fceumm-rom` option adds the optional Gun Smoke reference lane.

Hostname refresh follow-up - 6 September 2026

Before moving controller hostname refresh out of the send path, twelve isolated `.local` lookups on the Controller cleared only that diagnostic process's resolver cache. Median lookup duration was 2.35 ms; the maximum was 107.54 ms. This is a small lookup sample, not an end-to-end camera-to-game latency measurement.

Controller sends now read a background-refreshed address. A blocked lookup does not block a send call or retain any controller samples; unavailable/expired addresses skip the current send. Deterministic tests stall resolution while attempting 100 states, then verify that only the next new state is sent after resolution completes. Capture, inference, movement thresholds, smoothing, native state publication, and core consumption are unchanged. The stationary-hand and synchronized physical display comparison remain pending.

Signed transport processing - 6 September 2026

An isolated comparison ran on the actual Controller app container and RetroPie with dummy input and no network transmission or gamepad publication. Each column uses 2,000 samples after 100 warm-up iterations. It compares the prior v1 codec/token check against the new v2 signing/challenge validation code loaded in memory; this is not a deployed camera-to-display test.

Stage and machine	Protocol	Median ms	p95 ms	Maximum ms
Packet creation, Controller	v1	0.1550	0.2142	0.4525
Packet creation, Controller	v2	0.2654	0.3364	2.4920
Validation, RetroPie	v1	0.1090	0.1473	0.3143
Validation, RetroPie	v2	0.4488	0.5640	0.7606

The representative released-state packet grew from 505 to 606 bytes, using a dummy 32-character token. Signed packet size no longer depends on token length. Median processing additions were 0.1104 ms for Controller packet creation and 0.3398 ms for RetroPie validation. These microbenchmarks do not include scheduling, network transit, initial handshake, uinput/native publication, core consumption, display latency, or stationary jitter. No movement smoothing or recognition settings changed. The synchronized physical movement baseline remains pending.

Native latency and stationary-jitter session

The development tools below prepare the next physical session. No new live latency or stationary-jitter result is claimed. Keep the current recognition, camera, calibration, smoothing, and video settings unchanged for the baseline.

Preflight and camera placement

Record the installed software/core identities, active player, calibrated state, selected native-state path, and effective RetroArch video settings (including per-game append files). Confirm `lr-nestopia-powerglove`, device `517`, and that the Robo-Glove follows the hand. Local send success alone is not receiver acceptance. Keep lighting, player position, and game conditions fixed. Close the Controller preview for the primary baseline.

Put the external camera behind and slightly to one side of the player, looking past the shoulder. Both the real hand and the cabinet screen must be visible; the hand must not obscure the Robo-Glove. Keep them at similar vertical positions in the recording where practical to reduce rolling-shutter timing differences. Leave the PowerGlove Vision Controller camera in its normal playing position. Make a short framing clip before starting measurements.

An iPhone high-frame-rate original is useful: 120 fps gives 8.3 ms frame spacing, 240 fps gives 4.2 ms. A Mac camera works too; verify its actual recording cadence (30 fps gives 33.3 ms spacing). One recording supplies the timeline for both events, so no phone/Mac clock synchronization is necessary. Preserve the original file. Slow-motion playback time is not necessarily physical elapsed time.

Guided windows

Run from the development checkout on the Mac:

```
SH
python3 scripts/run-native-latency-session.py \
  --status-url http://UNO-Q-NAME.local:8088/status \
  --output-dir /tmp/powerglove-session-01
```

The read-only runner waits for Enter before each window, counts down, then collects status at a requested 50 ms interval. It guides three 20-second supported open-hand holds, then ten moves in each direction: five short and five longer steps, with a hold and return to center between moves. Each direction lasts 60 seconds. Terminal cues pace the operator; they do not synchronize clocks. A direct worker status endpoint through an SSH tunnel can avoid the supervisor cache; it still samples rather than recording every inference.

Reports include observed-sample gaps, tracking-loss transitions, X/Y span and standard deviation, and active button sample counts during neutral windows. These counts are observations, not complete gesture-event counts. A stationary candidate needs continuous observed detection/calibration and no active buttons; video, delivery, condition changes, and request errors must still be reviewed. Repeat invalid holds using the single-window collector and a new output path. Physical tremor is part of the live measurement, not isolated tracker noise.

Optional correlated software tracing

Tracing is disabled by default. It adds no controller packets and does not change the signed transport or the 64-byte native-state ABI. Set these variables in the **environment of the processes being started**, not just a later SSH shell:

```
SH
POWERGLOVE_DIAGNOSTIC_TRACE=/tmp/pgv-session-01
POWERGLOVE_DIAGNOSTIC_SECONDS=180
```

On the Controller, the App Lab supervisor passes its environment to the worker. On RetroPie, use a temporary receiver service environment override. Start with existing launch arguments, ports, private token files, and configuration. Do not start a second worker or receiver alongside the normal one. Verify the selected environment reached the actual worker/receiver. Use separate windows or up to 600 seconds when preparing a longer session; the duration starts at process initialization, so allow time for launch and preflight. Remove temporary overrides and restart normally afterward. Normal installation does not enable tracing.

Each process reserves a new private file named `PREFIX.ROLE.PID.json`. It retains at most 20,000 events in memory. Recording uses a nonblocking lock, drops evidence under contention, and freezes when its time/capacity limit is reached. A background thread exports the frozen window; normal close also requests export. Gameplay callbacks perform no file writes. A crash/forced termination can leave incomplete evidence: only parse finalized files. `dropped` and `stop_reason` describe loss or early truncation; counts stop when the window freezes. Files contain timing, hashed session correlation, sequence, X/Y and button flags, but no images, landmarks, token, raw session identifier, address, or player name.

Controller events separate camera-read completion, processing start, tracking completion, gesture/calibration completion, and encode/send start/end. Receiver events record userspace socket return, validation completion, publication start and completion, plus the published record's timestamp/guard. These are local clock measurements; camera exposure/driver buffering and kernel socket arrival remain outside their boundaries.

Core consumption needs a **separate diagnostic build** in a fresh build directory:

```
SH
POWERGLOVE_BUILD_DIAGNOSTICS=1 \
  scripts/build-nestopia-powerglove.sh build/nestopia-latency-01
```

Build on the target architecture; a Mac `.dylib` cannot run on RetroPie. The result is named `nestopia_powerglove_diagnostic_libretro`, distinct from the normal core. The production patch and its digest remain unchanged. Select the diagnostic binary only for the test launch, preserving all existing arguments and the `POWERGLOVE_NATIVE_STATE` path. Do not replace the installed normal core. Set `POWERGLOVE_CORE_DIAGNOSTIC_TRACE=/tmp/pgv-core-01.csv` and the same duration in RetroArch's launch environment. Leave the old verbose `POWERGLOVE_TRACE` unset. The diagnostic callback buffers at most 20,000 consumption records, with no logging or disk writes in the callback. **Exit the game normally** to export the CSV; it is not readable as complete evidence until unload. Saturation is reported in its final `# dropped=` line. Restore the normal core selection afterward.

Collect matching files from one session. Receiver and core must use the same cabinet boot and native-state path. Controller/receiver joins use a hashed session and sequence; receiver/core joins use sequence, guard, and publication timestamp, so profile/session sequence resets do not falsely match older publications.

```
SH
python3 scripts/analyze-latency-trace.py \
  --controller /tmp/pgv-controller.json \
  --receiver /tmp/pgv-receiver.json --core /tmp/pgv-core-01.csv \
  --same-cabinet-boot --output /tmp/pgv-stages.json
```

The report counts first observed consumption per publication. Repeated core reads are not additional input samples. Missing matches can reflect window boundaries, overwritten states, or dropped evidence; they are not automatically network loss. Network transit and physical display latency remain explicitly unmeasured here. Never subtract independent monotonic clocks or add stage percentiles. Measure clock offset and its uncertainty separately before attempting one-way network attribution; no such synchronization is implemented by these tools.

Instrumentation overhead

Run `scripts/benchmark-diagnostic-overhead.py --output /tmp/pgv-overhead.json` with the appropriate Python interpreter on each device. It needs no camera, network traffic, or virtual gamepad. It compares 5,000 iterations after warm-up with the trace guard disabled/enabled, including event allocation, timestamps, session hashing, and buffer insertion. It excludes core tracing and file export.

A preparation run on the development Mac (arm64, Python 3.14.7) measured disabled p50/p95 0.042/0.083 microseconds and enabled 0.833/0.917 microseconds, with an enabled maximum of 1,301.875 microseconds. This is a local synthetic measurement, not a device or gameplay result. Before using traced gameplay results, repeat workload windows with tracing off/on/off on the actual devices, including the normal/diagnostic core comparison. Report distribution changes and capture/ tracking continuity; do not silently subtract a synthetic overhead estimate.

Original-video review and screenshots

Install `av` (PyAV) and `Pillow` into a temporary **Mac diagnostic environment**; they are not Controller or receiver runtime dependencies. First index the original recording and extract selected zero-based frames for visual review:

```sh
python scripts/analyze-latency-video.py --video /path/to/original.mov \
  --frames 100,101,102 --output-dir /tmp/pgv-video-index
```

The report includes the original SHA-256 and decoded presentation timestamp of every frame, bounded to 150,000 frames. Inspect candidate onset frames and their immediate predecessors. Record first physical motion and first corresponding Robo-Glove motion, not terminal cue time. To measure stopping, also mark hand stop and game settling. Use a reviewed annotation file with this structure (example frame numbers and digest are placeholders, not measurements):

```json
{
  "video_sha256": "digest-from-index-report",
  "timing_verified": true,
  "seconds_per_pts_second": 1,
  "trials": [{
    "label": "right-1", "direction": "right", "unoccluded": true,
    "hand_onset": 100, "game_onset": 120,
    "hand_stop": 130, "game_settled": 150
  }],
  "stationary": []
}
```

Set `timing_verified` only after confirming actual capture cadence and retiming. The scale is real seconds per file-timestamp second; use 1 for an original whose PTS already represent physical time. Only a verified uniform retime can use a single different scale. Gaps, duplicate/backward timestamps, and intervals over 1.5 times the median are rejected for measurement. A recording with changing slow-motion speed needs an original uniform-timing export, not a guessed scale.

```sh
python scripts/analyze-latency-video.py --video /path/to/original.mov \
  --annotations /tmp/pgv-annotations.json \
  --output-dir /tmp/pgv-video-analysis
```

The analyser reports onset median/p95/range, each onset's preceding-frame timing bracket, per-direction counts, and optional stop-to-settle time. It extracts annotated PNG stills without covering the original picture. These screenshots are evidence from the recording, not remotely timed screenshots of the display. Exposure, rolling shutter, and annotation

uncertainty remain beyond the frame bracket and must be stated with conclusions.

Optional `trajectory` points on a trial contain `frame`, `hand_x`, `hand_y`, `glove_x`, and `glove_y` in original-image pixels. Supply at least three chronological points covering one movement through its settled endpoint. The report compares normalized progress and reports glove overshoot; these are following-behavior measures, not another latency estimate. Optional `stationary` entries contain `label`, `unoccluded`, `tracking_losses`, and `points` with the same five fields. They report sampled hand/screen coordinate span and standard deviation separately; tracking loss or occlusion prevents acceptance. Positions are manually reviewed, not inferred automatically. Use matching telemetry to identify tracking loss and unintended gestures, and sample enough of each full 20-second hold to describe resting behavior.

A synthetic 100 fps video with a known ten-frame delay produced 100 ms onset, with a 90-110 ms frame-sampling bracket; extracted stills were visually checked. This validates the analysis path, not the physical cabinet. Keep raw recordings, traces, indexes and individual coordinates temporary and local. Retain aggregate reports and selected non-sensitive annotated evidence in this benchmark record only after the physical session. Tune one identified stage at a time, requiring repeatable responsiveness improvement without increased resting movement or reduced recognition reliability.

UNO Q Kiyo Pro capture comparison - September 6, 2026

After deploying comfortable reach support, an isolated capture experiment used the UNO Q's installed worker Python/OpenCV environment and its attached Kiyo Pro. The normal camera worker was idle and paused during capture, with controller output stopped. Each sequential lane warmed up for two seconds and measured six seconds. No hand images were saved and no inference workload was run. All lanes requested MJPEG at 60 fps; OpenCV reported accepting the requested buffer counts. Actual FPS uses completed frames, not the negotiated value. Every lane had zero failed reads.

Camera controls	Image size	Buffers	Actual fps	Read interval p50 / p95 (ms)	Decode p50 / p95 (ms)
Original HDR state unknown	640×480	1	25.78	35.39 / 67.42	16.56 / 17.20
Original HDR state unknown	640×480	2	30.00	32.60 / 51.67	17.63 / 18.68
Original HDR state unknown	1280×720	1	23.25	35.95 / 71.20	21.57 / 28.03
Original HDR state unknown	1280×720	2	30.09	32.13 / 53.04	21.29 / 21.57
HDR-off, auto exposure, fixed rate	640×480	1	51.68	16.24 / 32.16	11.86 / 16.35
HDR-off, auto exposure, fixed rate	640×480	2	59.71	16.02 / 27.80	10.29 / 12.05
HDR-off, auto exposure, fixed rate	1280×720	1	21.90	45.74 / 64.10	22.91 / 23.21
HDR-off, auto exposure, fixed rate	1280×720	2	52.43	16.11 / 36.19	15.55 / 15.98

Camera controls	Image size	Buffers	Actual fps	Read interval p50 / p95 (ms)	Decode p50 / p95 (ms)
HDR-off, auto exposure, fixed rate	640×480	2	59.76	16.06 / 21.82	10.37 / 12.12
HDR-off, auto exposure, fixed rate	1280×720	2	55.20	15.81 / 42.27	15.57 / 17.16

The 480p/two-buffer/HDR-off candidate delivered 59.71 and 59.76 fps. Increasing to 720p cost more decoding time and produced 52.43 and 55.20 fps with longer tails. The selected experimental candidate therefore retained 640×480, requested two buffers, and explicitly requests volatile HDR-off, automatic exposure, and fixed frame rate. It used the then-current two inference threads and independent latest-frame capture. These remain historical experiment conditions. Version 0.4.0 uses four threads and Automatic camera rate, which prefers 30 fps with driver fallback; it does not make the vendor-specific HDR command a general default.

Standard format/exposure controls were restored after the isolated experiment, and the worker was resumed. The original HDR state could not be read; HDR-off was sent without an onboard SAVE command. The experiment compares sequential configurations; it does not establish HDR's isolated contribution, exposure-to-read freshness, recognition under load, or physical hand-to-screen latency. Those still require live play and a hand/display recording. The protocol reference is kiyoproctrls; UNO code uses a bounded, USB-identity-checked control implementation without adding that external utility. Aggregate source evidence is retained locally in `data/benchmarks/uno-q-camera-2026-09-06.json` (excluded from installation payloads).

Direct V4L2 and lean-graph comparison - September 7, 2026

The later output-paused Controller comparison used the same Kiyo Pro at 640×480 MJPEG and 30 fps, with low-latency automatic exposure and the volatile Kiyo HDR-off request. The direct V4L2 reader initialized without fallback, reported advancing camera sequence numbers, and supplied monotonic driver timestamps. A fresh observed frame measured about 1.25 ms from its driver timestamp to userspace dequeue, and the capture interval was the expected 33.3 ms. This establishes compatibility for the tested camera and Controller; it does not assume every V4L2 driver exposes the same ABI, format, or timestamp.

Matched twelve-second status windows then compared the complete MediaPipe Hands output graph with the configuration-only `lean-image` output set. The strongest steady segments measured:

Graph	Observed samples	Inference p50 / p95 (ms)	Sample age p50 / p95 (ms)
Complete	18	43.9 / 91.8	81.4 / 108.5
Lean image output	23	43.2 / 84.4	70.0 / 114.6

The lean lane reduced median inference by only about 1.6% and increased p95 sample age. It therefore failed the required 10% end-to-end promotion gate. The complete graph was restored; direct V4L2 remains available as the selected camera-specific test option. A later complete-graph confirmation contained a single 215 ms tail, reinforcing that short status windows should guide rather than replace longer gameplay and physical latency validation.